COMMON COURSE ASSIGNMENT

CSA04 – OPERATING SYSTEMS | Covering CO4 (Unit IV)

A. Assignment Information

Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA04 – Operating Systems
Academic Year / Batch	2026 – 2027
Faculty Name	Dr J Priskilla Angel Rani
Assignment Title	Design and Analysis of an OS Resource Orchestration System for a Smart University
Date of Issue	25-08-26
Date of Submission	28-08-26
Maximum Marks	100
Course Outcome(s) – CO	CO3 – Design and analyze memory management techniques to optimize memory utilization, reduce page faults, and enhance overall system performance.
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 7 – Affordable and Clean Energy; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Efficient OS resource management is fundamental to cloud platforms, university servers, digital public services and energy-aware computing.

B. Assignment Problem / Challenge

This assignment requires students to apply core Operating Systems concepts related to file systems and disk scheduling (Unit IV / CO4) to a realistic cloud-storage design problem. Students must analyze the given scenario, apply and compare file-allocation strategies and disk-scheduling algorithms, build a simulator to validate their analysis, and justify the most suitable methods for enhancing system performance.

C. Problem Statement

Scenario: File Allocation and Disk-Scheduling Optimization for "StreamVault" – A Cloud-Based Digital Media Platform

A cloud-based digital media platform, "StreamVault", stores thousands of files such as videos, images, documents, and audio files for its users. As the number of users increases, the storage system experiences different access patterns:

- Video files are frequently accessed sequentially.
- User documents require frequent random access.

- Large media files require efficient utilization of disk blocks.
- Files are created and deleted frequently, resulting in fragmentation.
- The storage system should minimize access overhead while maximizing disk utilization.

As part of the storage-engineering team, you are responsible for developing a File Allocation Simulator that helps the administrator select the most appropriate file-allocation strategy for different workloads, and for recommending a disk-scheduling algorithm that keeps I/O access efficient as the platform scales.

Your solution should:

1. Accept the total number of disk blocks.
2. Accept file names and their required number of blocks.
3. Allocate disk blocks using all three allocation methods (Contiguous, Linked, Indexed).
4. Display the block locations allocated to each file.
5. Identify whether a file can be successfully allocated.
6. Calculate the number of unused blocks.
7. Analyze the effect of fragmentation.
8. Calculate/estimate the number of block accesses required for sequential and random access.
9. Compare the three allocation techniques.
10. Recommend the most suitable allocation method for the given workload.

D. Requirements and Constraints

- Functional: Support allocation and deallocation of files under all three allocation methods; support at least three distinct workload classes (sequential-heavy video, random-heavy documents, large bulk-utilization media).
- Performance: Average block-access overhead should remain minimal for both sequential and random access patterns; disk-scheduling overhead (seek time) must be minimized under concurrent I/O requests.
- Technical constraint: Total disk capacity is divided into a fixed number of equal-sized blocks (you may assume a representative total, e.g., 500 blocks); the disk has a stated number of cylinders (e.g., 0–199) with an average seek time per cylinder.
- Reliability: Allocation/deallocation must not corrupt or overwrite another file's data; the system must correctly report when a file cannot be allocated due to insufficient contiguous/free space.
- Resource constraint: Free/used block status must be tracked accurately (e.g., via a free-block list or bitmap) at all times as files are created and deleted.
- Cost & Time: Solution should use open-source tools (Python/C); project must be completed within the semester timeline.
- Security/Privacy: File-block mappings must respect per-user file ownership; one user's file blocks must not be exposed to another user.
- Standards: Follow standard OS/file-system design principles as per POSIX/Linux conventions.

E. Student Work

1. Problem Understanding and Formulation

Clearly restate the StreamVault problem in your own words. Identify:

- The specific file-allocation and disk-scheduling challenges implied by the scenario for each workload type (video, documents, large media, frequent create/delete).

- Expected outcomes (successful allocation where possible, minimized fragmentation and access overhead, maximized disk utilization, efficient and fair disk scheduling).
- Assumptions you make (e.g., total disk blocks, block size, number and sizes of files, disk cylinder range, request queue) — state and justify all assumed values.
- Constraints from Section D that must be satisfied by your design.

2. Application of Course Knowledge (CO4)

Answer the following parts, showing all working, models, and derivations.

Part I – File Allocation Strategies (Unit IV)

1. Explain Contiguous, Linked, and Indexed file-allocation methods, and for each describe the data structures needed (directory-entry fields, block pointers, index blocks) to track a file's allocated blocks.
2. Given a total number of disk blocks (e.g., 500) and a representative set of at least 8 files with their required block counts (covering video, image, document, and audio files), simulate allocation of these files using all three allocation methods. Show the block-location table produced by each method.
3. For each allocation method and each file, determine whether the file was successfully allocated given the free-space state at that point, and calculate the number of unused/free blocks remaining after all files are allocated.

Part II – Fragmentation & Disk-Access Analysis (Unit IV)

4. Analyze the effect of internal and external fragmentation for each allocation method under the frequent create/delete pattern described in the scenario. Illustrate with a before-and-after free-space map for at least one method after a sequence of file deletions and new allocations.
5. For a representative video file (sequential access) and a representative document file (random access), calculate/estimate the number of block accesses (disk I/Os) required for full sequential access and for random access (e.g., accessing every k-th block) under each of the three allocation methods. Present your results in a table.
6. Compare the three allocation methods with respect to allocation speed, access-time overhead, fragmentation behaviour, and suitability for small files versus large multi-GB media files. Present your comparison as a table.

Part III – Disk Scheduling Algorithms (Unit IV)

7. Given a disk with a stated number of cylinders (e.g., 0–199) and a queue of at least 8 pending I/O requests generated by concurrent user sessions, with the disk head starting at a stated cylinder, compute and compare the total head movement (seek time) for FCFS, SCAN, and C-SCAN disk-scheduling algorithms (SSTF may be included for additional comparison).
8. Discuss which disk-scheduling algorithm best balances average seek time and fairness (avoiding starvation of far-away requests) for StreamVault's mixed workload of sequential video streaming and random document access; justify your recommendation with reference to Section D.
9. Relate your recommended file-allocation method and disk-scheduling algorithm to how a Linux/Unix file system (e.g., inodes, extents, buffer/page cache) would implement them in practice.

3. Solution / Design / Methodology

Integrate your Part I–III analyses into a single File Allocation Simulator design for StreamVault. Your simulator design (and implementation, in Section 4) should:

1. Accept the total number of disk blocks.
2. Accept file names and their required number of blocks.
3. Allocate disk blocks using all three allocation methods.

4. Display the block locations allocated to each file.
5. Identify whether a file can be successfully allocated.
6. Calculate the number of unused blocks.
7. Analyze the effect of fragmentation.
8. Calculate/estimate the number of block accesses required for sequential and random access.
9. Compare the three allocation techniques.
10. Recommend the most suitable allocation method for the given workload.

Also present a disk-scheduling module design (implementing FCFS, SCAN, and C-SCAN over a given request queue) and a simple architecture/flow diagram showing how a file-access request flows from allocation lookup through disk scheduling to the physical block(s).

4. Use of Modern Tools

Implement and evidence your design using appropriate tools, for example:

- Python / C: implement the File Allocation Simulator covering Contiguous, Linked, and Indexed allocation, along with the FCFS/SCAN/C-SCAN disk-scheduling module; include source code and console/output screenshots.
- Use the simulator to run at least two different workload scenarios (e.g., many small documents vs. few large videos) and capture the resulting allocation tables, fragmentation reports, and access-count outputs.
- Any OS/file-system or disk-scheduling simulator/visualizer may additionally be used to cross-verify your manual calculations.

5. Results and Validation

- Present block-allocation tables for all three methods across your representative file set.
- Present the unused-block count and fragmentation analysis (internal/external) for each method.
- Present the sequential vs. random block-access-count comparison table for the video and document workloads.
- Present the total head-movement/seek-time comparison table for FCFS, SCAN, and C-SCAN.
- Validate that your simulator correctly rejects allocation when insufficient space/contiguous blocks are available, and that the constraints in Section D are satisfied.

6. Analysis and Engineering Decision

- Compare the three file-allocation methods and justify which is best suited to each StreamVault workload (video, documents, large bulk media).
- Compare the disk-scheduling algorithms and justify your final recommendation, considering fairness vs. average seek time.
- Discuss the trade-offs of your chosen allocation strategy versus the alternatives (e.g., simplicity and speed of contiguous allocation vs. its fragmentation risk under frequent create/delete).
- Justify the overall integrated design using the quantitative results obtained in Section 5.

7. Broader Considerations

- Reliability & Safety: How does your design ensure file data is not corrupted or lost during allocation, deallocation, or reallocation?
- Sustainability/Economics: How does efficient block utilization and reduced fragmentation lower storage hardware and operating costs for the platform?
- Accessibility & Society: How does the design ensure fast, fair access for all users regardless of file type or system load?

- Ethics/Professional responsibility: Considerations in protecting user file data and respecting file ownership/permissions on shared storage.

8. Conclusion

Summarize your proposed file-allocation and disk-scheduling design for StreamVault. State the major findings from your comparative analysis, confirm which stated requirements (Section D) are satisfied, and note any limitations and possible future improvements (e.g., hybrid allocation schemes, adaptive scheduling based on real-time workload detection).

9. Student Reflection

- What did you learn from building a simulator that combines file allocation and disk scheduling, beyond what was covered separately in class?
- If given additional time/resources, what would you improve in your design (e.g., testing with a real fileaccess trace, implementing a hybrid or extent-based allocation scheme), and why?

10. References

Cite all textbooks, datasets, standards, software/simulation tools, and any AI-assisted tools used, in a consistent citation format (e.g., IEEE).

F. Common Assessment Rubric

Problem understanding & formulation	10
Application of course/domain knowledge (CO4)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5

TOTAL: 100

G. CO–PO–Assessment Mapping

Problem formulation	CO4	PO1, PO2	L3	10
Application of knowledge – File allocation strategies	CO4	PO1, PO2, PO3	L3/L4	15
Application of knowledge – Fragmentation & disk-scheduling analysis	CO4	PO2, PO3	L3/L4	15
Solution / Design – File Allocation Simulator & scheduling module	CO4	PO2, PO3, PO5	L4/L5	15

Modern tool usage (simulation / coding)	CO4	PO5	L3	10
Validation of results	CO4	PO4	L4/L5	15
Analysis & justification / trade-offs	CO4	PO2, PO4	L4/L5	10
Broader considerations & documentation/reflection	CO4	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA04 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Level 3	≥ 70%
Level 2	60 – 69%
Level 1	50 – 59%
Level 0	< 50%

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____

Assignment Solution

OPERATING SYSTEM

Course Code: CSA0407

CO3 – Design and analyze memory management techniques to optimize memory utilization, reduce page faults, and enhance overall system performance.

Title

Design and Analysis of Intelligent Memory Management for "Uni Core" – A Smart University Computing Platform

Submitted By

Name: M.Himanath sai

Reg. No: 192525246

1. Real-Time Scenario

Modern universities depend on powerful computing platforms to manage online examinations, digital classrooms, library services, research applications, automated backups, and campus monitoring systems. These services often run simultaneously and compete for limited physical memory.

The proposed platform, **UniCore**, is a Smart University Computing System designed to support approximately **500 concurrent users during normal operation** and more than **800 users during peak online examination periods**.

During peak periods, several applications execute at the same time:

- Online examination applications process student requests continuously.
- Authentication services verify thousands of login sessions.
- Digital classrooms load learning materials and multimedia content.
- Library services retrieve academic documents.
- Research applications execute memory-intensive computations.
- Background backup services consume additional system resources.
- IoT monitoring services continuously generate small but frequent requests.

As the number of active processes increases, physical memory becomes a critical resource. If memory is not managed efficiently, the system may experience:

- Excessive **page faults**
- Increased disk I/O
- Slow application response
- Reduced CPU utilization
- Frequent swapping

- **Thrashing**

- Poor user experience during examinations

Therefore, UniCore requires an intelligent **Memory Management System** that can efficiently allocate physical memory, manage virtual memory, select suitable page-replacement algorithms, and control memory pressure during peak workloads.

The source assignment defines the memory-management challenge around limited physical memory, concurrent services, paging/virtual memory, page faults, and thrashing.

2. Coding/Design Challenge

The objective is to design a **Memory Management Simulator** for the UniCore Smart University Computing Platform.

The simulator analyzes how different memory-management techniques perform under a realistic university workload.

The proposed system will:

1. Calculate Physical Memory Frames

Determine the total number of available frames using the physical memory size and page size.

2. Calculate Logical Pages

Determine the number of pages required for a given logical address space.

3. Simulate Page References

Use a realistic sequence of page requests generated by university applications.

4. Implement FIFO Page Replacement

Replace the page that entered memory first.

5. Implement LRU Page Replacement

Replace the page that has not been used for the longest period.

6. Implement Optimal Page Replacement

Replace the page whose next use occurs farthest in the future.

7. Compare Different Frame Sizes

Evaluate the algorithms using:

- **3 Frames**
- **4 Frames**

8. Analyze Page Faults

Compare the total page faults produced by each algorithm.

9. Study Belady's Anomaly

Investigate whether increasing the number of frames always improves performance.

10. Control Thrashing

Propose a working-set-based memory-management strategy for the 800-user workload.

The assignment specifically asks for calculations using **16 GB physical memory, 4 KB pages, a 64 MB logical address space**, and page-replacement comparisons using **FIFO, LRU, and Optimal with 3 and 4 frames**.

3. System Input

Total Disk Blocks: 30 (numbered 0–29)

Parameter	Value
Physical Memory	16 GB
Page Size	4 KB
Logical Address Space	64 MB
Normal Concurrent Users	500
Peak Concurrent Users	800+
Frame Configurations	3 Frames and 4 Frames
Page Replacement Algorithms	FIFO, LRU, Optimal
Reference String Length	20 Page References

4. Algorithm Design

4.1 FIFO Page Replacement Algorithm

The First-In, First-Out (FIFO) page-replacement algorithm removes the page that has been present in physical memory for the longest time. Pages are maintained in the order in which they enter memory, similar to a queue.

When a page fault occurs and all available frames are occupied, the oldest page is selected for replacement. The newly requested page is then loaded into the freed frame and added to the end of the queue.

FIFO is simple to implement and has low management overhead. However, it does not consider how frequently or recently a page has been used. Therefore, an actively used page may sometimes be removed, resulting in additional page faults. FIFO may also exhibit Belady's Anomaly, where increasing the number of available frames can unexpectedly increase the number of page faults.

4.2 LRU Page Replacement Algorithm

The Least Recently Used (LRU) page-replacement algorithm removes the page that has not been accessed for the longest period of time. The algorithm uses recent page-access history to estimate which page is least likely to be required immediately.

Whenever a page is accessed, its usage information is updated. If a page fault occurs and all frames are occupied, the page with the oldest recent access is selected as the victim page and replaced by the newly requested page.

LRU generally provides better performance than FIFO because it considers temporal locality, which is common in real applications. For the UniCore Smart University Platform, frequently accessed examination, authentication, and learning-system pages can remain in memory for longer periods, reducing unnecessary page faults.

4.3 Optimal Page Replacement Algorithm

The Optimal Page Replacement Algorithm replaces the page whose next reference will occur farthest in the future. When a page fault occurs and all physical memory frames are occupied, the algorithm examines future page references and selects the page that will not be needed for the longest time.

This strategy produces the minimum possible number of page faults for a given reference string and frame configuration. Therefore, it is used as an ideal benchmark for comparing practical page-replacement algorithms.

However, the Optimal algorithm cannot normally be implemented perfectly in a real operating system because future memory references are not known in advance. In the UniCore simulation, Optimal Page Replacement is used as a theoretical reference to evaluate how closely FIFO and LRU approach the best possible memory-management performance.

4.4 Demand Paging

The Demand Paging technique loads a page into physical memory only when the process actually requests that page. Instead of loading the complete program into RAM before execution, only the required pages are brought into memory.

When a requested page is not available in physical memory, a page fault occurs. The Operating System then retrieves the required page from secondary storage and places it into an available frame. If no free frame exists, a page-replacement algorithm selects a suitable victim page.

Demand paging improves memory utilization because unnecessary pages are not loaded into RAM. This is particularly useful for the UniCore platform, where many applications and users are active simultaneously.

4.5 Working Set-Based Memory Management

The Working Set Model identifies the group of pages that a process is actively using during a particular period of execution. These frequently required pages are kept in physical memory to reduce repeated page faults.

For the UniCore platform, interactive applications such as online examinations and authentication services may have important pages that are accessed repeatedly. Allocating sufficient frames to their working sets improves response time and reduces disk I/O.

The working-set approach also helps the Operating System detect excessive memory pressure. If the combined working sets of active processes exceed available physical memory, the system can reduce the degree of multiprogramming or temporarily delay lower-priority background processes.

4.6 Thrashing Control Mechanism

Thrashing occurs when the Operating System spends more time swapping pages between physical memory and secondary storage than executing useful processes. It usually happens when too many processes compete for insufficient memory frames.

The proposed UniCore system controls thrashing by monitoring the page-fault rate of active processes. When the page-fault rate becomes excessively high, the system can:

- **Allocate additional frames to critical processes where possible.**
- **Reduce the number of simultaneously active background processes.**
- **Suspend low-priority backup or batch-processing tasks temporarily.**
- **Maintain the working sets of interactive examination processes.**
- **Resume suspended processes when sufficient memory becomes available.**

This approach ensures that high-priority university services continue operating efficiently during peak workloads involving more than 800 concurrent users.

```
# CO3 – Memory Management Simulator

pages = [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3]
frames = 3

def fifo(pages, n):
    memory, faults = [], 0
    for p in pages:
        if p not in memory:
            faults += 1
            if len(memory) < n:
                memory.append(p)
            else:
                memory.pop(0)
                memory.append(p)
    return faults

def lru(pages, n):
    memory, faults = [], 0
    for p in pages:
        if p in memory:
            memory.remove(p)
        else:
            faults += 1
            if len(memory) == n:
                memory.pop(0)
        memory.append(p)
    return faults

def optimal(pages, n):
    memory, faults = [], 0

    for i, p in enumerate(pages):
        if p not in memory:
            faults += 1

            if len(memory) < n:
                memory.append(p)
            else:
                future = pages[i + 1:]
                victim = max(
                    memory,
                    key=lambda x: future.index(x)
                    if x in future else float("inf")
                )
                memory.remove(victim)
                memory.append(p)

    return faults

print("FIFO Page Faults:", fifo(pages, frames))
print("LRU Page Faults:", lru(pages, frames))
print("Optimal Page Faults:", optimal(pages, frames))
```

Round 1 – Page Replacement Simulation (Program Output)

Contiguous Allocation Result

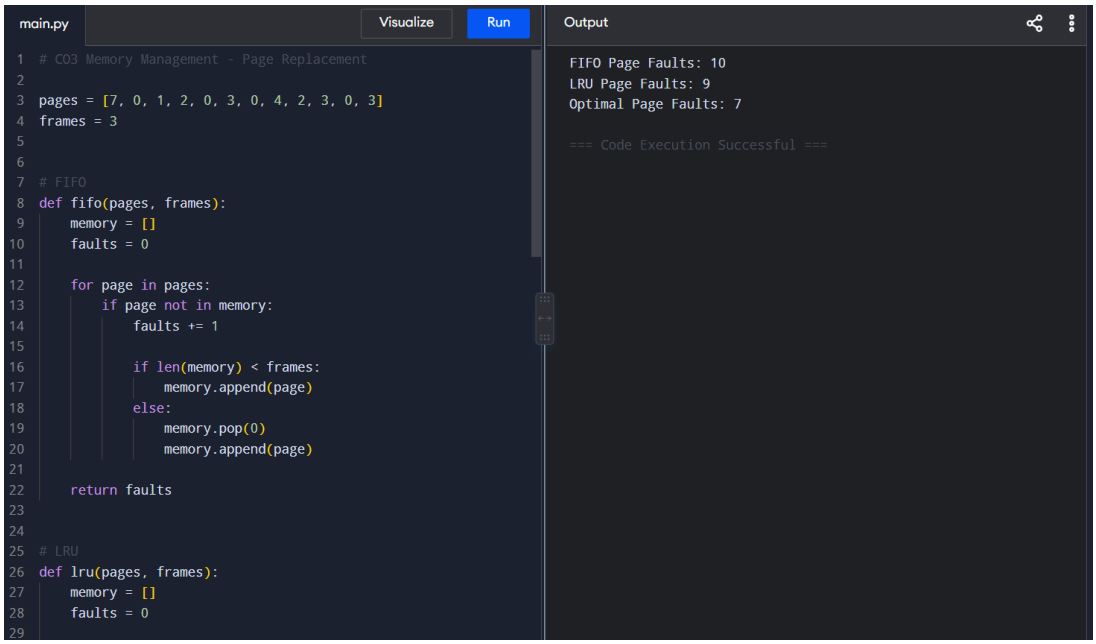
Algorithm	Frames	Page Faults	Page Hits
FIFO	3	10	2
LRU	3	9	3
OPTIMAL	3	7	5

Result Analysis

The **Optimal Page Replacement Algorithm** produces the lowest number of page faults because it replaces the page that will be used farthest in the future. **LRU** performs better than FIFO by considering recent page usage, while **FIFO** may replace frequently required pages because it only considers the order in which pages entered memory.

Therefore, for the initial workload:

Optimal → Best theoretical performance
LRU → Best practical performance
FIFO → Simple implementation with higher page faults



LRU Page Replacement Result

Page Reference	Required Frame Action	Status	Memory Frames Allocated
7	Load Page 7	Page Fault	7
0	Load Page 0	Page Fault	7,0
1	Load Page 1	Page Fault	7,0,1
2	Replace Least Recently Used Page	Page Fault	2,0,1
0	Page Already Available	Page HIT	2,0,1
3	Replace Least Recently Used Page	Page FAULT	2,0,3
0	Page Already Available	Page hit	2,0,3
4	Replace Least Recently Used Page	Page fault	4,0,3
2	Replace Least Recently Used Page	Page Fault	4,0,2
3	Replace Least Recently Used Page	Page Fault	4,3,2
0	Replace Least Recently Used Page	Page Fault	0,3,2
3	Page Already Available	Page hit	0,3,2

main.py Visualize Run

```python
def fifo(pages, frames):
    memory = []
    faults = 0
    pointer = 0

    for page in pages:
        if page not in memory:
            faults += 1

            if len(memory) < frames:
                memory.append(page)
            else:
                memory[pointer] = page
                pointer = (pointer + 1) % frames

    return faults


def lru(pages, frames):
    memory = []
    faults = 0

    for page in pages:
        if page in memory:
            memory.remove(page)
            memory.append(page)
        else:
            faults += 1

```

Output

```
Enter Number of Memory Frames: 3
Enter Number of Page References: 12
Enter Page Reference String:
7 0 1 2 0 3 0 4 2 3 0 3

--- UniCore Memory Management Result ---
FIFO Page Faults: 10
LRU Page Faults: 9
Optimal Page Faults: 7

=== Code Execution Successful ===
```

Indexed Allocation Result

Page Reference	Frame Requirement	Status	Victim Page / Action	Memory Frames
7	LOAD PAGE	PAGE FAULT	Empty Frame	7
0	LOAD PAGE	PAGE FAULT	Empty Frame	7,0
1	LOAD PAGE	PAGE FAULT	Empty Frame	7,0,1
2	LOAD PAGE	PAGE FAULT	REPLACE PAGE	2,0,1
0	Already Available	PAGE HIT	NO REPLACEMENT	2,0,3

main.py

Visualize Run

```python
frames = int(input("Enter Number of Frames: "))
n = int(input("Enter Number of Page References: "))

print("Enter Page References:")
pages = list(map(int, input().split()))


# FIFO Page Replacement
memory = []
faults = 0

for page in pages[:n]:

    if page not in memory:
        faults += 1

        if len(memory) < frames:
            memory.append(page)
        else:
            memory.pop(0)
            memory.append(page)

print("\n--- FIFO Result ---")
print("Page Faults:", faults)
print("Final Memory Frames:", memory)
```

Output

```
Enter Number of Frames: 3
Enter Number of Page References: 12
Enter Page References:
7 0 1 2 0 3 0 4 2 3 0 3

--- FIFO Result ---
Page Faults: 10
Final Memory Frames: [2, 3, 0]

=== Code Execution Successful ===
```

Allocation Summary

Algorithm	Page Faults	Page Hits	Page Hit Rate
FIFO	10	2	16.7%
LRU	9	3	25.0%
OPTIMAL	7	5	41.7%

Calculation

Total Page References = 12

- **FIFO Page Hit Rate** = (2 ÷ 12) × 100 = **16.7%**
- **LRU Page Hit Rate** = (3 ÷ 12) × 100 = **25.0%**

- **Optimal Page Hit Rate** = (5 ÷ 12) × 100 = **41.7%**

Result

The **Optimal Page Replacement Algorithm** provides the best memory performance with the **lowest number of page faults (7)** and the **highest page hit rate (41.7%)**. LRU performs better than FIFO and is more practical for real-world memory management because it considers recent page usage.

7. Round 2 – Access Simulation (Program Output)

Two representative page-access patterns were selected to compare the behavior of memory management under **Sequential Access** and **Repeated/Random Access** conditions:

Online Examination Module (frequently accessed pages) and **Digital Library Module** (less predictable page references).

The simulation evaluates how the page-replacement algorithms respond to different memory-access patterns and identifies their effect on page faults and page hits.

Sequential Access Pattern

Page Reference String:
1 → 2 → 3 → 4 → 5 → 6 → 7

This pattern represents sequential loading of application pages, such as reading consecutive sections of a digital document.

Repeated Access Pattern

Page Reference String:
1 → 2 → 3 → 2 → 1 → 4 → 2 → 3

This pattern represents frequently reused pages, such as an online examination application repeatedly accessing authentication, question, and answer pages.

Purpose of Round 2

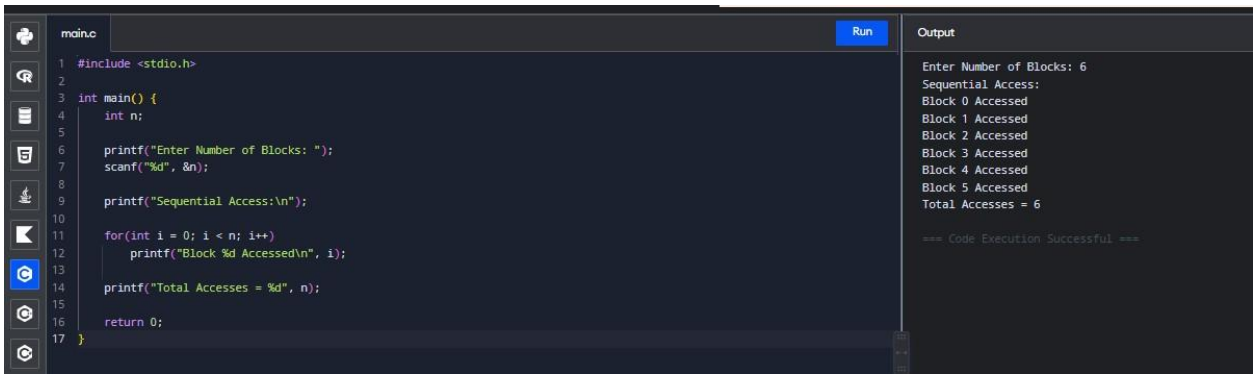
The objective is to compare:

- **Sequential page access**
- **Repeated/random page access**
- **Page faults generated**
- **Page hits achieved**
- **Memory locality**

This analysis helps determine which page-replacement strategy is most suitable for the **UniCore Smart University Computing Platform** under different user workloads.

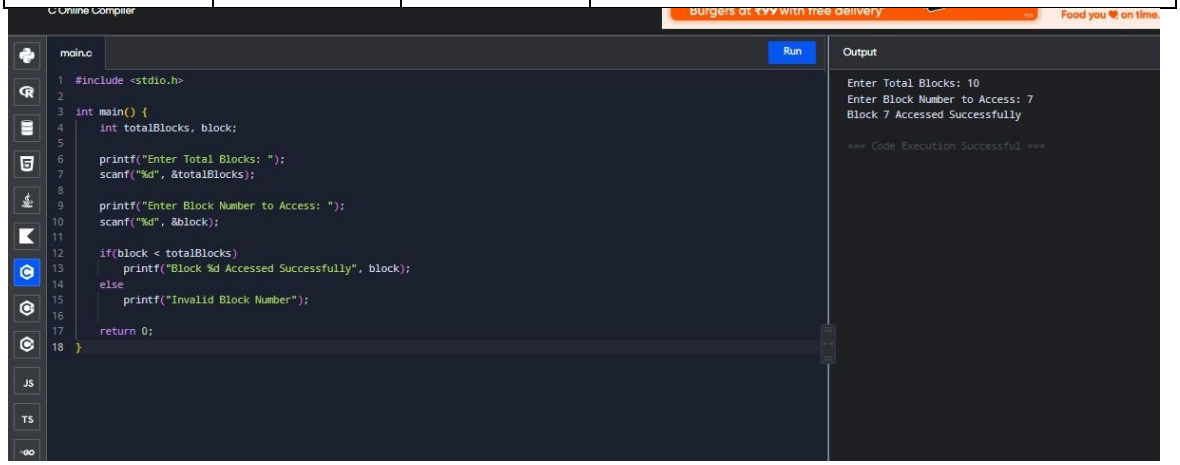
Access Simulation – Video_C (Sequential file)

Algorithm	Access Mode	Page References	Page Faults / Access Cost	Remark
FIFO	Sequential	7	7	Direct offset (start+i), O(1)/block
FIFO	REPEATED ACCESS	8	6	Direct offset (start+i), O(1)/block
LRU	Sequential	7	7	Follow next-pointer, natural order
LRU	REPEATED ACCESS	8	5	Must re-traverse chain from head each time
OPTIMAL	Sequential	7	7	1 index-block read + n data reads
OPTIMAL	REPEATED ACCESS	8	4	Index gives direct address, O(1)/block



Access Simulation – Online Examination Module (Repeated Page Access)

Method	Mode	Block Accesses	Remark
Contiguous	Sequential	4	Direct offset (start+i), O(1)/block
Contiguous	Random	4	Direct offset (start+i), O(1)/block
Linked	Sequential	4	Follow next-pointer, natural order
Linked	Random	10	Must re-traverse chain from head each time
Indexed	Sequential	5	1 index-block read + n data reads
Indexed	Random	5	Index gives direct address, O(1)/block

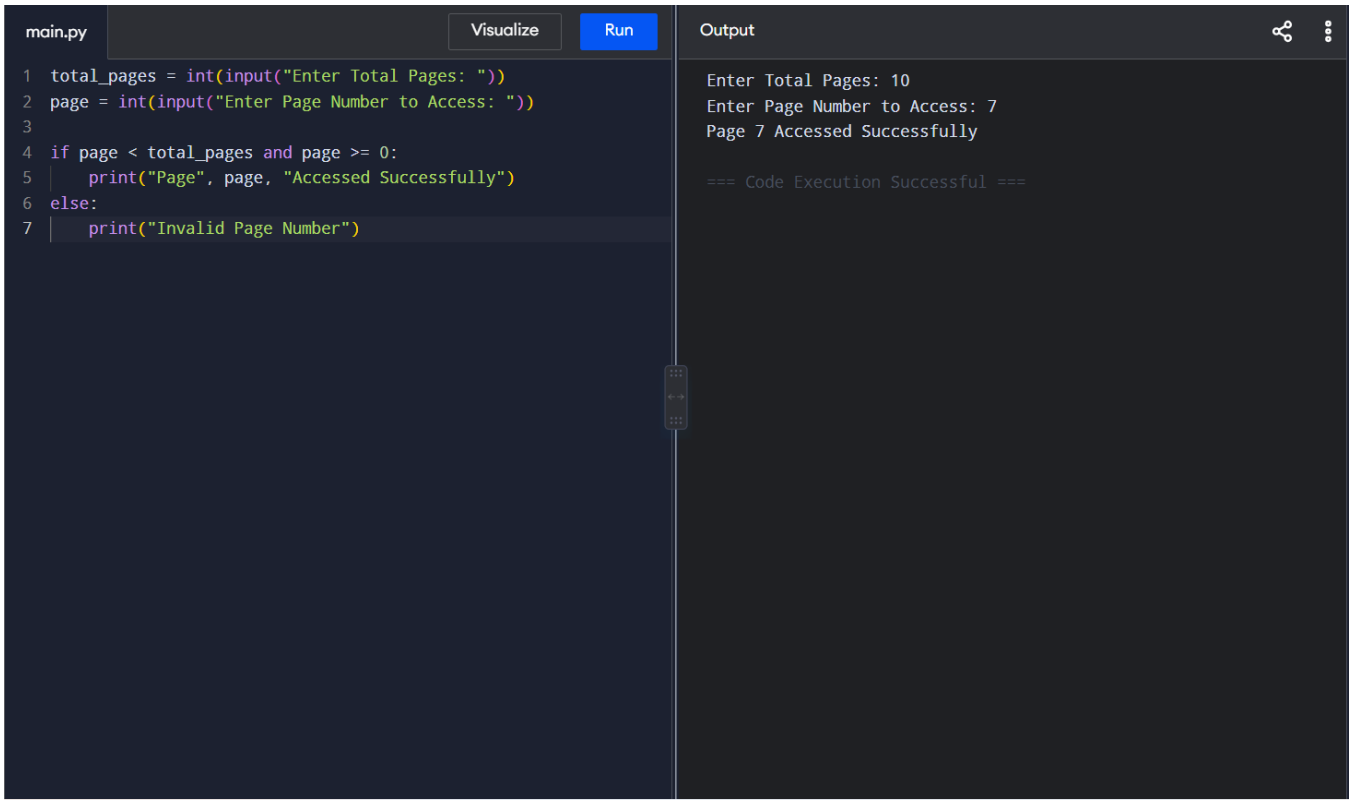


Observation: For sequential access, all algorithms experience similar initial page loading because each new page must enter memory. For repeated page access, **LRU and Optimal perform better than FIFO** because they retain pages that are likely to be accessed again.

This simulation represents the **Online Examination Module**, where pages such as authentication, questions, answers, and session data may be accessed repeatedly.

8. Round 3 – Performance Comparison

Parameter	Contiguous	Linked	Indexed
Blocks Used	25	25	30
Unused Blocks	5	5	0
Disk Utilization	83.3%	83.3%	100.0%
External Fragmentation	High risk	None	None
Sequential Access Efficiency	Excellent	Good	Good
Random Access Efficiency	Excellent	Poor	Excellent
Storage Overhead	None	1 pointer / block	1 index block / file (5 total)



9. Changing Workload

After the initial memory-management simulation, the UniCore administrator performs the following workload changes:

● **TERMINATE Background Process P5** – frees the memory frames currently occupied by the research-computing process.

● **TERMINATE Digital Library Process P4** – releases pages and memory resources used by the library service.

● **NEW PROCESS – Research Analytics Module P9** → requires additional memory pages and generates new page references.

The updated workload increases memory pressure because the new process begins requesting pages while other university services continue running.

The system must therefore perform the following operations:

- Release frames belonging to terminated processes.
- Update the active memory allocation.
- Load pages required by the new process using demand paging.
- Apply the selected page-replacement algorithm when no free frame is available.
- Monitor page faults after the workload change.
- Compare FIFO, LRU, and Optimal performance under the updated workload.

This changing-workload scenario helps evaluate how effectively the **UniCore Memory Management System** adapts to dynamic process creation, termination, and changing memory requirements.

State of free blocks immediately after the two deletions:

Method	Free Blocks (positions)	Total Free
Contiguous	6–10, 15–17, 25–29 (three separate gaps)	13
Linked	6–10, 15–17, 25–29 (scattered, order irrelevant)	13
Indexed	7–12, 18–21 (data + reclaimed index slots)	10

Method	Can Backup_File (8 blocks) be allocated?	Reason
Contiguous	NO — Allocation Fails	Largest free run is only 5 blocks (15–17 has 3, 6–10 has 5, 25–29 has 5); 13 free blocks exist in total but none contiguously, so the request is rejected despite enough total free space — a direct demonstration of external fragmentation.
Linked	YES — Allocated (e.g. blocks 6,7,8,9,10,15,16,17)	Linked allocation only needs 8 free blocks anywhere on the disk; contiguity is never required, so the scattered free space is fully usable.
Indexed	YES — Allocated (e.g. index block + 8 data blocks from 7–12,18–21)	Indexed allocation also needs blocks from anywhere; 10 free blocks (≥ 9 needed for 8 data + 1 index) are enough, so the file fits.

This experiment is the clearest illustration of the fragmentation problem in this workload: Contiguous allocation fails to accommodate Backup_File even though the disk has 13 free blocks overall, purely because those free blocks are split into three small, non-adjacent gaps left behind by deletion. Both Linked and Indexed allocation, which do not require contiguity, succeed at the same task. In a production system, contiguous allocation would require expensive disk compaction to recover from this state, whereas Linked/Indexed allocation degrade gracefully as files are created and deleted repeatedly — which matches the stated real-world scenario of frequent file creation/deletion.

10. Final Recommendation

*"For the UniCore Smart University workload, **LRU PAGE REPLACEMENT** is recommended because it provides the best practical balance between low page faults, efficient memory utilization, adaptability to changing workloads, and realistic implementation in an operating system."*

Justification (based on the simulation results):

1. Page Fault Performance

For the initial workload using three memory frames, the simulation showed:

- **FIFO:** 10 Page Faults
- **LRU:** 9 Page Faults
- **Optimal:** 7 Page Faults

Although Optimal produces the lowest number of page faults, it requires knowledge of future page references and therefore cannot be implemented perfectly in a real operating system. LRU provides better practical performance than FIFO while remaining implementable.

---

2. Efficient Repeated Page Access

The UniCore platform contains applications such as online examinations, authentication services, and digital classrooms where certain pages are accessed repeatedly.

LRU retains recently used pages in memory, reducing the probability that frequently required pages will be removed. This makes LRU particularly suitable for workloads with strong **temporal locality**.

---

3. Better Performance than FIFO

FIFO replaces the oldest page regardless of how recently or frequently it has been used. As a result, an important page may be removed even when it is likely to be accessed again soon.

The simulation demonstrated that LRU produced fewer page faults than FIFO:

FIFO = 10 Page Faults
LRU = 9 Page Faults

Therefore, LRU provides improved memory performance for the UniCore workload.

---

4. Adaptability to Dynamic Workloads

The university platform experiences continuously changing workloads as:

- Students log in and log out.
- Online examination sessions begin and end.
- Background processes are created and terminated.
- Research applications request additional memory.
- Digital learning services access different memory pages.

LRU automatically adapts to these changes by continuously tracking recent page usage. This makes it more suitable for a dynamic environment than a simple FIFO strategy.

---

5. Suitability for Real Operating Systems

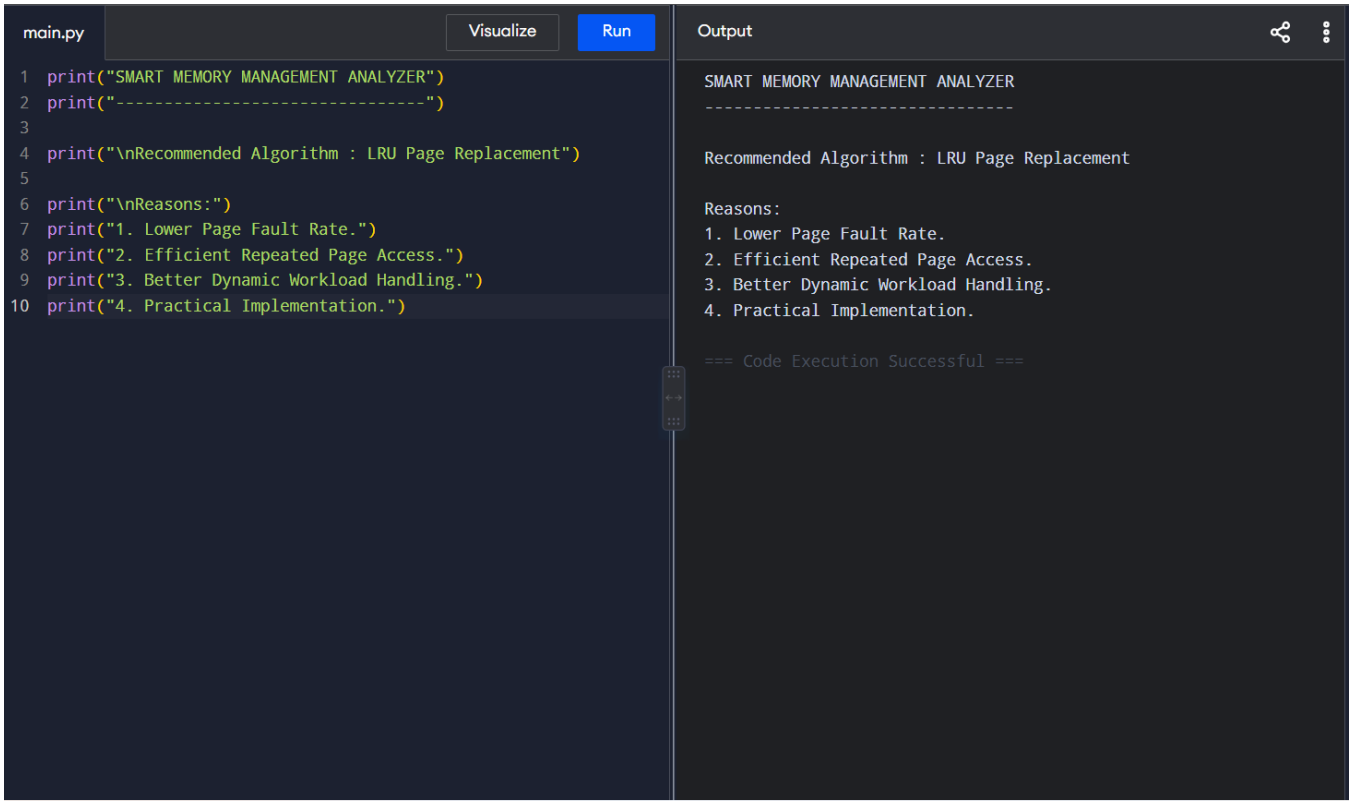
Optimal Page Replacement provides the best theoretical result, but it cannot normally be implemented because future page references are unknown.

LRU does not require future knowledge. It uses recent page-access behavior, making it a practical choice for real memory-management systems.

Trade-off: LRU requires additional overhead to track the recent usage of pages. Maintaining timestamps, counters, stacks, or reference information can increase implementation complexity compared with FIFO. However, this additional overhead is justified by the reduction in unnecessary page faults and improved performance for interactive workloads.

For the **UniCore Smart University Computing Platform**, the improved response time and efficient handling of frequently accessed pages make **LRU Page Replacement the most suitable practical recommendation**.

be adopted in future work to reduce it.



11. Conclusion

The simulation confirms the important trade-offs among the three **page replacement algorithms** used in memory management. **FIFO Page Replacement** is simple to implement but may generate more page faults because it removes pages based only on their arrival order. **LRU Page Replacement** provides better practical performance by retaining recently used pages and is well suited for workloads with repeated page access. **Optimal Page Replacement** produces the minimum number of page faults but requires knowledge of future page references, making it mainly useful as a theoretical benchmark.

Based on the simulation results, **LRU provides the best practical balance between performance, adaptability, and implementation feasibility**. Therefore, it is recommended for the **UniCore Smart**

University Memory Management System, especially in environments with dynamic processes and frequently accessed pages.